

Projeto: Transferência Confiável de Arquivos com UDP

1. Objetivo do Projeto

Neste projeto, a equipe deverá desenvolver um sistema de transferência de arquivos utilizando o protocolo UDP.

O UDP é um protocolo rápido, porém não garante que os dados cheguem corretamente, nem que cheguem na ordem certa. Por isso, a equipe deverá implementar mecanismos próprios para tornar a comunicação confiável.

O sistema deverá permitir o envio, listagem e download de arquivos entre cliente e servidor, utilizando somente sockets UDP.

2. Linguagens Permitidas

O projeto poderá ser desenvolvido em uma das seguintes linguagens:

C, C++, C#, Java ou Python.

3. Equipes

O projeto poderá ser feito individualmente, em dupla ou em trio, com no máximo 3 integrantes.

A equipe deverá ser formada pelos mesmos integrantes do Projeto 1.

4. Estrutura do Sistema

O sistema deverá ser dividido em duas partes:

Servidor: responsável por receber, armazenar, listar e enviar arquivos.

Cliente: responsável por permitir que o usuário envie arquivos, visualize arquivos disponíveis e faça download.

A comunicação entre cliente e servidor deverá ser feita obrigatoriamente com UDP.

5. Funcionalidades do Servidor

O servidor deverá:

1. Ficar em execução aguardando requisições dos clientes.
2. Receber arquivos enviados pelos clientes.
3. Salvar os arquivos recebidos em uma pasta específica, por exemplo: arquivos_recebidos.
4. Enviar ao cliente a lista de arquivos disponíveis no servidor.
5. Enviar arquivos para download quando solicitado pelo cliente.

6. Atender mais de um cliente, utilizando multithreading ou outra estratégia adequada à linguagem escolhida.

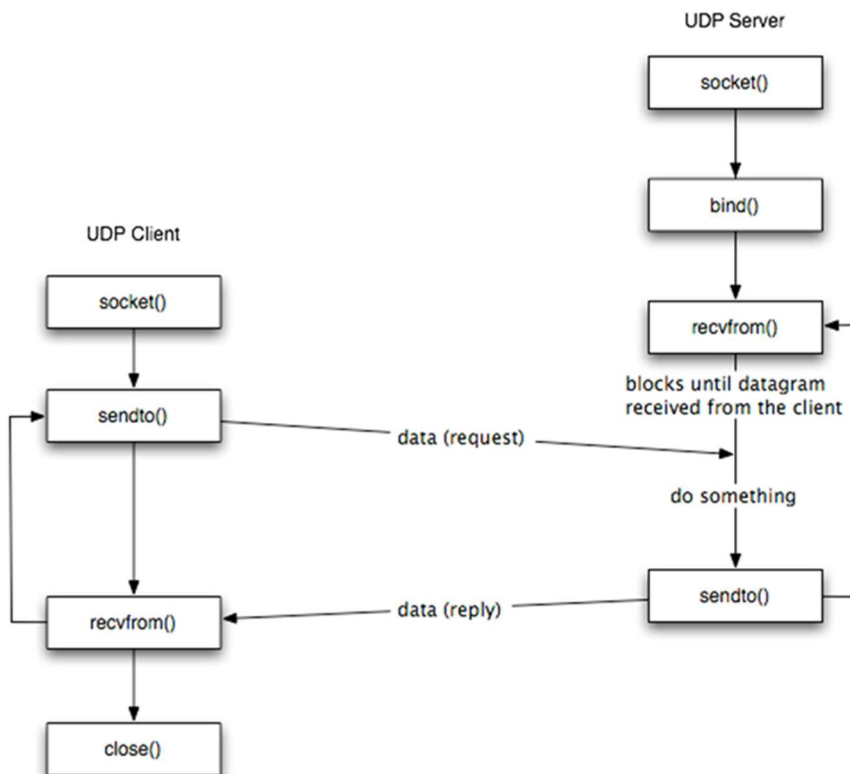
6. Funcionalidades do Cliente

O cliente deverá permitir que o usuário:

1. Envie um arquivo do computador para o servidor.
2. Solicite e visualize a lista de arquivos disponíveis no servidor.
3. Escolha um arquivo disponível e faça o download para o computador.

7. Confiabilidade sobre UDP

Como o UDP não possui confiabilidade automática, a equipe deverá implementar essa lógica manualmente.



O sistema deverá ter:

7.1 Numeração dos pacotes

Arquivos grandes deverão ser divididos em pacotes menores.

Cada pacote deverá possuir um número de sequência, por exemplo:

pacote 1, pacote 2, pacote 3, pacote 4...

Essa numeração será usada para remontar o arquivo corretamente no destino.

7.2 Garantia de ordem

O sistema deverá garantir que os pacotes sejam organizados na ordem correta antes da reconstrução do arquivo final.

Mesmo que os pacotes cheguem fora de ordem, o arquivo reconstruído deve ficar correto.

7.3 Confirmação de recebimento com ACK

O receptor deverá enviar uma confirmação de recebimento, chamada ACK, para indicar que determinado pacote foi recebido corretamente.

Exemplo:

O cliente envia o pacote 3.

O servidor recebe o pacote 3.

O servidor responde com um ACK confirmando o recebimento do pacote 3.

7.4 Retransmissão por timeout

Se o remetente enviar um pacote e não receber o ACK dentro de um tempo limite, deverá considerar que o pacote foi perdido.

Nesse caso, o pacote deverá ser enviado novamente.

Essa situação deverá ser exibida no console durante os testes.

8. Estratégia de Envio

A equipe deverá escolher uma das estratégias abaixo:

8.1 Stop-and-Wait

Nesta estratégia, o remetente envia um pacote e espera o ACK antes de enviar o próximo.

Funcionamento:

1. Envia um pacote.
2. Aguarda o ACK.
3. Se receber o ACK, envia o próximo pacote.
4. Se não receber o ACK dentro do tempo limite, retransmite o pacote.

Essa estratégia é mais simples, porém mais lenta.

8.2 Sliding Window

Nesta estratégia, o remetente pode enviar vários pacotes antes de receber todos os ACKs.

Funcionamento:

1. Define uma janela de envio.
2. Envia vários pacotes dentro dessa janela.
3. Recebe os ACKs dos pacotes enviados.
4. Avança a janela conforme os ACKs são recebidos.
5. Retransmite os pacotes perdidos quando necessário.

Essa estratégia é mais eficiente, porém mais complexa.

Material de apoio:

https://www.tkn.tu-berlin.de/teaching/rn/animations/gbn_sr/

9. Regras Importantes

É proibido utilizar sockets TCP.

É proibido utilizar bibliotecas prontas que já implementem controle de ordem, ACK, retransmissão ou confiabilidade automática.

A equipe deverá implementar a própria lógica de:

Numeração de pacotes.

Controle de ACK.

Timeout.

Retransmissão.

Ordenação dos pacotes.

Reconstrução do arquivo.

Códigos iguais ou muito semelhantes entre equipes serão considerados plágio e poderão receber nota zero.

10. Testes Obrigatórios

A equipe deverá testar e demonstrar:

1. Envio de arquivo do cliente para o servidor.
2. Listagem dos arquivos disponíveis no servidor.
3. Download de arquivo do servidor para o cliente.

4. Reconstrução correta do arquivo no destino.
5. Simulação de perda de pacotes.
6. Retransmissão dos pacotes perdidos.
7. Ordenação correta dos pacotes.
8. Exibição no console dos ACKs, timeouts e retransmissões.

Para simular perda de pacotes, a equipe poderá implementar no código uma condição que descarte pacotes aleatoriamente.

Exemplo:

O receptor ignora um pacote propositalmente.

O remetente não recebe o ACK.

O timeout é acionado.

O pacote é retransmitido.

O receptor recebe o pacote corretamente.

11. Uso do Wireshark

A equipe deverá utilizar o Wireshark para capturar e analisar os pacotes UDP da aplicação.

No vídeo, a equipe deverá mostrar a captura dos pacotes e relacionar com os conceitos vistos em aula, como:

Camada de enlace.

Camada de rede.

Camada de transporte.

Endereço IP.

Porta UDP.

Datagrama UDP.

Envio e recebimento de pacotes.

12. O que Entregar

A equipe deverá entregar:

12.1 Código-fonte da aplicação

O código deverá estar completo, organizado e comentado.

A entrega deverá conter:

Código do servidor.

Código do cliente.

Arquivos necessários para execução.

Instruções básicas para executar o projeto.

12.2 Vídeo de apresentação

A equipe deverá gravar um vídeo demonstrando o funcionamento do sistema e explicando a implementação.

O vídeo deverá mostrar a aplicação funcionando de verdade, com execução do cliente e do servidor.

No vídeo, a equipe deverá explicar:

1. Qual linguagem foi utilizada.
2. Como o servidor funciona.
3. Como o cliente funciona.
4. Como é feito o upload de arquivos.
5. Como é feita a listagem de arquivos.
6. Como é feito o download de arquivos.
7. Qual estratégia foi utilizada: Stop-and-Wait ou Sliding Window.
8. Como os pacotes foram numerados.
9. Como os ACKs foram implementados.
10. Como o timeout foi configurado.
11. Como ocorre a retransmissão dos pacotes perdidos.
12. Como o sistema garante a ordenação dos pacotes.
13. Como o arquivo é reconstruído corretamente.
14. Como foi feita a simulação de perda de pacotes.
15. Como o Wireshark comprova a comunicação via UDP.

Todos os integrantes da equipe deverão participar do vídeo, explicando alguma parte do projeto.